UMassAmherst | Manning College of Information & Computer Sciences

Programming Methodology

Lab 5: Midterm 1 Review

Wednesday, March 4, 2026

Weekly Lab Agenda

- Go over reminders/goals
- Review past material
- Work in groups of 2-3 to solve a few exercises.
    - Please sit with your group from last week.
- Discussion leaders will walk around and answer questions.
- Solutions to exercises will be reviewed as a class.
- Attendance will be taken at the end.

Reminders

- Midterm 1 is tonight
  - Good luck everyone! :)

Today's Goals

- Midterm Review

Exam Logistics + Advice

- Location & Time: **<u>7-9pm</u>**, see below for location (decided by *your labs start time*!)
  - **ILC S140**: **Section 2 labs of 9:05, 10:10, 11:15**
  - **ISB 135: rest of Section 2 (labs of 12:20, 1:25), all of Section 1**
- Do not prioritize doing the exam in order!
  - The order of the questions means nothing. Move on to the next question if you are stuck.
- Do not leave any question blank!
  - Don't write garbage or throw away comments. Write down notes of what you know, that might jog your memory.
- Write notes to yourself or underline important statements!
  - What does this function take in? What it returns? What is being graded?
- Ask questions!
  - If something doesn't make sense, raise your hand and a proctor will come over to help out.
- Use a pencil! If you don't have any, you can get them free at the library!

Topic Poll

- Raise your hand for your top 2-3 topics
  - Type signatures
  - Higher-order functions
  - Mental models
  - Lists
  - Testing

Type Signatures

Type Signatures

What are the types of g, a, and x?

```
const g = (a, x) => a.map(f => (x => f(f(x)))).map(f => f(x));
```

- Look at the smallest atomic parts of this function.
- Write down what you know immediately.
- Deduce the rest using a combination of what you figured out previously.

Type Signatures – Solution

```
const g = (a, x) => a.map(f => (x => f(f(x)))).map(f => f(x));
```

- a is an `Array`
  - For now we will say its a generic array, we might later figure out what it is (**`a: T[]`**)
- `.map` iterates over all elements of a
  - The `f`'s in the expression `.map(f => (x => f(f(x))))` are the elements of a
  - They are being called → they are functions
    - They take in the same type they return
  - **`a: ((x: T) => T)[]` `x: T`**
- `a.map(f => (x => f(f(x))))` is an array of functions of type `(x: T) => T`
  - `.map(f => f(x))` must return an array, calling the functions made in the previous expression will result in T
- Answer: **`g<T>(a: ((x: T) => T)[], x: T): T[]`**

Higher-Order Functions

Higher Order Functions (HOFs)

```
function reducer(acc, e) {
    return {
        x: acc.y + e,
        y: acc.x
    };
}
const pair = a => a.reduce(reducer, {x: "", y: ""});
```

What is **`pair(["that", "is", "a", "short", "text"])`**?

```
function reducer(acc, e)
{
    return {
        x: acc.y + e,
        y: acc.x
    };
}
```

reducer switches the value of x and y AND concatenates the current element e onto the value of x

index of array	["that", "is", "a", "short", "text"]	initialValue {x:"",y:""}
0th	acc={x:"",y:""} e = "that"	→ {x: "that", y:""}
1st	acc={x: "that", y:""} e = "is"	→ {x: "is", y:"that"}
2nd	acc={x: "is", y:"that"} e = "a"	→ {x: "that a", y: "is"}
3rd	acc= {x: "that a", y: "is"} e = "short"	→ {x: "is short", y: "that a"}
4th	acc={x: "is short", y: "that a"} e = "text"	

{x: "that a text", y: "is short"}

Mental Models + Closures

Mental Models

For the line defining o1 below, state how many objects except { } are created when executing that line. State what values are printed. Explain your answers.

```
function f1(k) {
  let o = {};
  while (--k >= 0) {
    o = {val: () => k, next: o}
  }
  return o;
}

let o1 = f1(3);
console.log(o1.val());
console.log(o1.next.val());
```

Mental Models

For the line defining o1 below, state how many objects except { } are created when executing that line. State what values are printed. Explain your answers.

```
1   function f1(k) {
2     let o = {};
3     while (--k >= 0) {
4       o = {val: () => k, next: o}
5     }
6     return o;
7   }
8
9   let o1 = f1(3);
10  console.log(o1.val());
11  console.log(o1.next.val());
```

The loop at line 3 executes three times; three objects are created and linked, next references the previous created object. The three `val` closures share the same environment and refer to the same variable k, they are identical.

Calling the closure `val` evaluates k; by this time, `f1` has completed and k is `-1`, this value is printed.

`o1.next.val` is an identical closure `() => k`. This prints `-1`, the value of k.

Mental Models

When `f1` starts executing, `k=3` is in the value map.

```
function f1(k) {
  let o = {};
  while (--k >= 0) {
    o = {val: () => k, next: o}
  }
  return o;
}

let o1 = f1(3);
console.log(o1.val());
console.log(o1.next.val());
```

Value

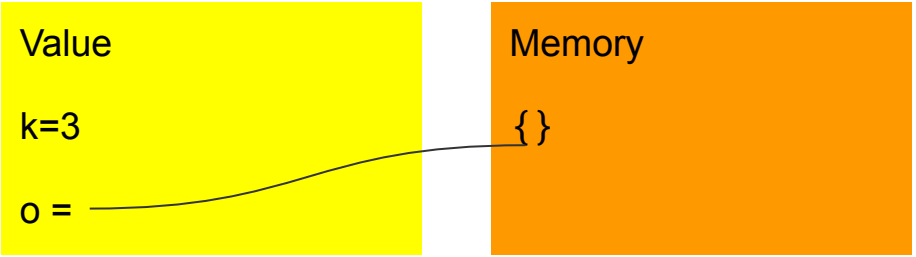
k=3

Memory

Mental Models

Line 2 creates an empty object is created in memory and stores its reference in o.

```
1   function f1(k) {
2     let o = {};
3     while (--k >= 0) {
4       o = {val: () => k, next: o}
5     }
6     return o;
7   }
8
9   let o1 = f1(3);
10  console.log(o1.val());
11  console.log(o1.next.val());
```

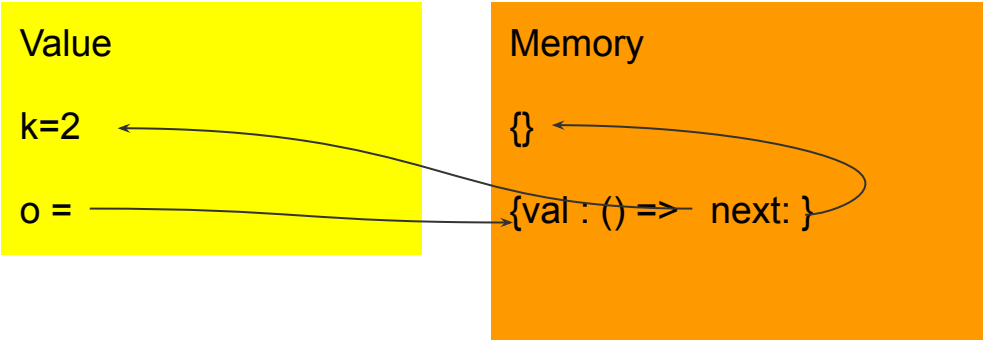


Mental Models

First iteration of the loop decreases value of k by 1 and creates a new object, next field of the newly created object copies reference in stored in o and o stores reference to the newly created object.

```
function f1(k) {
  let o = {};
  while (--k >= 0) {
    o = {val: () => k, next: o}
  }
  return o;
}

let o1 = f1(3);
console.log(o1.val());
console.log(o1.next.val());
```

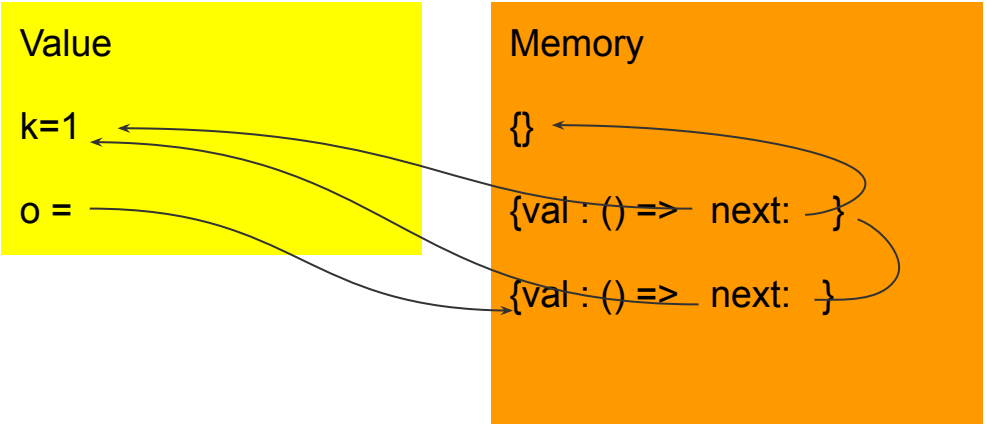


Mental Models

Second iteration of the loop another object is created.

```
function f1(k) {
  let o = {};
  while (--k >= 0) {
    o = {val: () => k, next: o}
  }
  return o;
}

let o1 = f1(3);
console.log(o1.val());
console.log(o1.next.val());
```

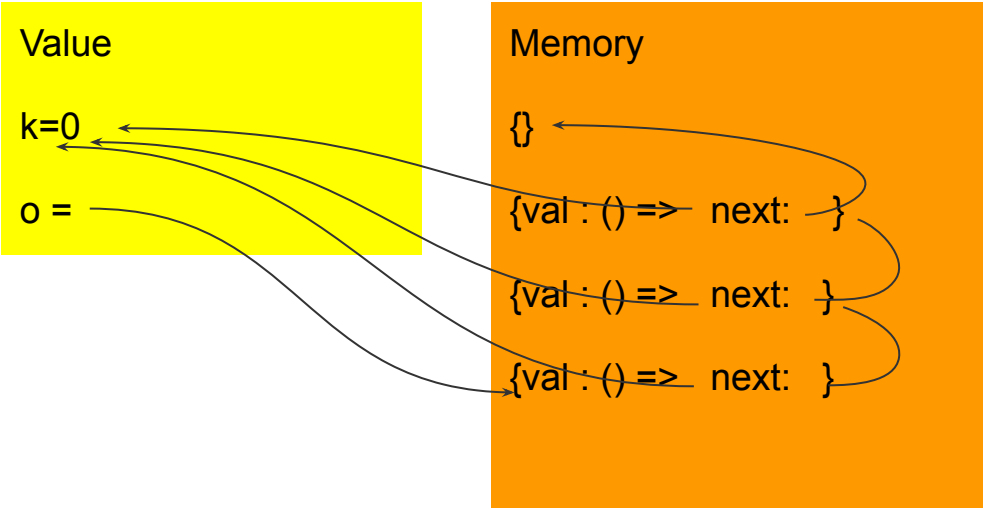


Mental Models

Third iteration of the loop. One more object created.

```
function f1(k) {
  let o = {};
  while (--k >= 0) {
    o = {val: () => k, next: o}
  }
  return o;
}

let o1 = f1(3);
console.log(o1.val());
console.log(o1.next.val());
```

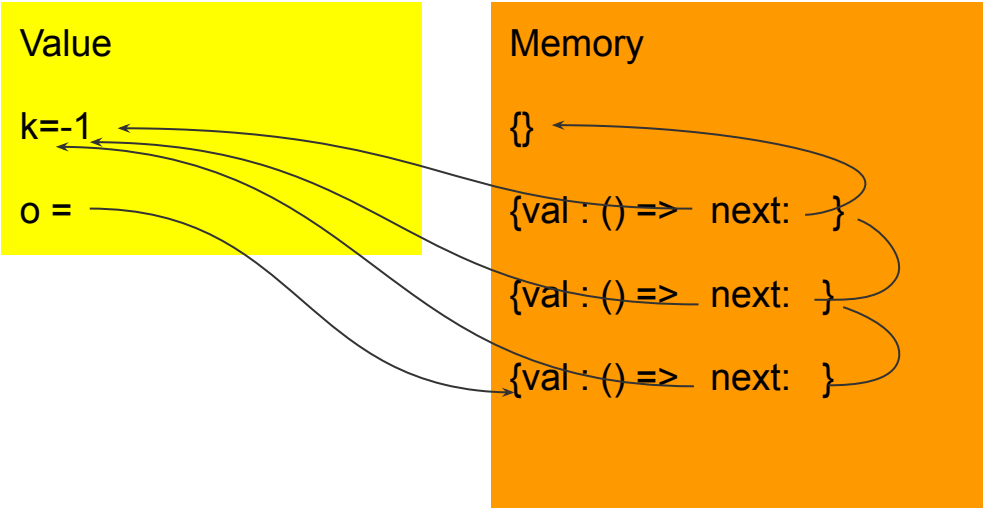


Mental Models

While condition is evaluated, which decreases k by 1. Body of the loop isn't executed.

```
function f1(k) {
  let o = {};
  while (--k >= 0) {
    o = {val: () => k, next: o}
  }
  return o;
}

let o1 = f1(3);
console.log(o1.val());
console.log(o1.next.val());
```

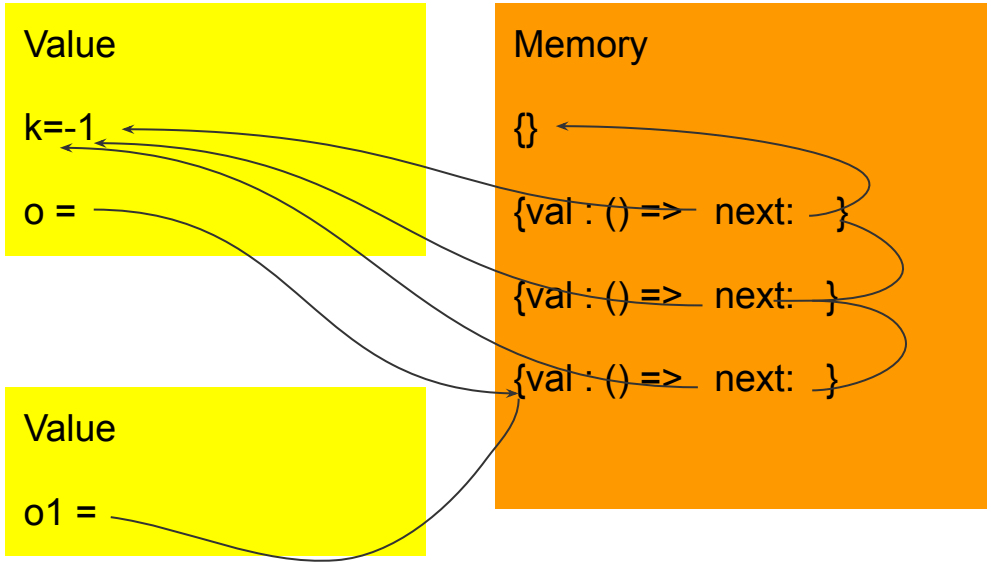


Mental Models

Upon return, o1 will have the reference value returned through o.

```
function f1(k) {
  let o = {};
  while (--k >= 0) {
    o = {val: () => k, next: o}
  }
  return o;
}

let o1 = f1(3);
console.log(o1.val());
console.log(o1.next.val());
```



Lists

Lists

Write a function **`sumEveryK(lst: List<number>, k: number): List<number>`** that splits **`lst`** into groups of **`k`** adjacent elements (the last group may have fewer) and returns a list with the sum of each group (in the same order). Assume **`k`** is a positive natural. Try not to create other temporary lists.

Example:

For `lst = 1,0,3,5,2,3,7` with `k = 3`

Output of `sumEveryK(lst, k)` should be `list = 4,10,7`

Lists - HOF Solution

```
function sumEveryK(lst: List<number>, k: number): List<number> {
  let sum = 0, rem = k;
  const result = lst.reduce((acc, elem) => {
    let output = acc;
    if (rem === 0) {
      output = node(sum, acc);
      sum = 0, rem = k;
    }
    rem--;
    sum += elem;
    return output;
  }, empty<number>());
  return reverseList(node(sum, result));
}
```

Lists - Recursive Solution

```
function sumEveryK(lst: List<number>, k: number): List<number> {
  // rem: count remaining to make group of k, acc: accumulated sum
  function sumRec(lst: List<number>, rem: number, acc: number): List<number> {
    // add last sum
    if (lst.isEmpty()) {
      return rem === k ? lst : node(acc, lst);
    }
    // set up recursion
    const newS = acc + lst.head()
    const tl = lst.tail();
    return --rem === 0 ? node(newS, sumRec(tl, k, 0)) : sumRec(tl, rem, newS);
  }
  return sumRec(lst, k, 0);
}
```

Testing

Testing

The `findIndex` function returns the index of the first element of array `a` for which `f` returns `true`, or `-1` if no such element exists

```
// findIndex<T>(a: T[], f: T => boolean): number
function findIndex(a, f) {
    for (let i = 0; i < a.length; ++i) {
        if (f(a[i])) { return i; }
    }
    return -1;
}
```

Write three representative cases that you would use to test `findIndex`.
You need not write code, but clearly indicate inputs, output, and the purpose of the test.

Testing

```
// findIndex<T>(a: T[], f: T => boolean): number
function findIndex(a, f) {
    for (let i = 0; i < a.length; ++i) {
        if (f(a[i])) { return i; }
    }
    return -1;
}
```

Three example tests:

`a: [ ], f: x => x > 0`. Output: `-1`. No element is found for an empty array.

`a: [4, 3, 2, 5], f: x => x % 2 > 0`. Output: `1`. Test first index is returned for multiple matches.

`a: ['hi', 'ho'], f: s => s.length > 2`. Output: `-1`. Test case when no element matches.

GOOD LUCK!!!

🤠